Smart Q&A — Complete AWS Deployment Plan

Course: Modern Software Deployment
Project: Smart Q&A — Course Discussion Platform
Region: us-east-1 (N. Virginia)
Stack: React 18 · FastAPI · PostgreSQL 15 · Docker · AWS

Table of Contents

1. Project Overview
2. Architecture Summary
3. AWS Services Used
4. Phase 1 — One-Time Infrastructure Setup
5. Phase 2 — CI/CD Automated Deployments
6. AI Similarity Search Feature
7. Environment Variables Reference
8. GitHub Secrets Reference
9. Deployment Timeline
10. Full Service Map

---

1. Project Overview

Smart Q&A is a full-stack course discussion platform (StackOverflow-style) where students post questions, submit answers, vote, and receive in-app notifications. It supports three user roles — Student, TA, and Admin — enforced through AWS Cognito groups.

Tech Stack

Layer	Technology
Frontend	React 18, Vite, Tailwind CSS, TanStack Query, React Router v6
Backend	Python 3.11, FastAPI, SQLAlchemy 2.0 (async), Alembic, Pydantic v2
Database	PostgreSQL 15 + pgvector extension
Authentication	AWS Cognito (JWKS-based JWT validation)
Containers	Docker, Docker Compose
CI/CD	GitHub Actions
AI Feature	Amazon Bedrock (Titan Embeddings) + pgvector

---

2. Architecture Summary

The application follows a **3-tier architecture** deployed entirely on AWS:

```
┌──────────────────────────────────────────────────────────────┐
│  TIER 1 — PRESENTATION                                       │
```

```
│  React SPA → Amazon CloudFront → Amazon S3                │
└──────────────────────────────────────────────────────────┘
                          │
                          │ HTTPS API calls /api/v1
                          ▼
┌──────────────────────────────────────────────────────────┐
│  TIER 2 — APPLICATION                                     │
│  App Load Balancer → EC2 #1 & EC2 #2 (FastAPI+Docker)  │
│  AWS Cognito (Auth) · Amazon SQS · AWS Lambda             │
│  Amazon Bedrock (AI Embeddings)                           │
└──────────────────────────────────────────────────────────┘
                          │
                          │ SQL / Vector queries
                          ▼
┌──────────────────────────────────────────────────────────┐
│  TIER 3 — DATA                                            │
│  Amazon RDS (PostgreSQL 15 + pgvector)                    │
│  Amazon S3 (File Attachments)                             │
└──────────────────────────────────────────────────────────┘
```

Authentication Flow

1. **Browser → Cognito** — User logs in; Cognito issues a signed JWT token
2. **Browser → CloudFront → ALB → EC2** — Every API request carries the JWT in the `Authorization` header
3. **EC2 → Cognito JWKS** — On first request, EC2 fetches Cognito's public keys and caches them in memory
4. **EC2 (local)** — All subsequent JWT verification is done locally using cached public keys — no per-request Cognito call

---

3. AWS Services Used

Service	Purpose	Tier
AWS Cognito	User authentication, JWT issuance, role groups	Application
Amazon CloudFront	CDN, HTTPS termination, routes API to ALB	Presentation
Amazon S3 (Frontend)	Hosts built React static files	Presentation
Application Load Balancer	Distributes traffic between EC2 instances, health checks	Application
Amazon EC2 (×2)	Runs FastAPI backend in Docker containers	Application
Amazon ECR	Private Docker image registry	CI/CD
Amazon RDS	PostgreSQL 15 database with pgvector extension	Data
Amazon S3 (Attachments)	Stores user-uploaded files, accessed via pre-signed URLs	Data
Amazon SQS	Decouples notification creation from processing	Application
AWS Lambda	Asynchronously processes notifications, writes to RDS	Application
Amazon Bedrock	Generates semantic text embeddings for AI similarity search	Application
GitHub Actions	CI/CD pipeline — test, build, deploy automatically	CI/CD

---

4. Phase 1 — One-Time Infrastructure Setup

These steps are performed **manually in the AWS Console once**, before automated deployments begin.

Step 1 — Security Groups (Networking)

Use the default VPC. Create three security groups with the following rules:

ALB Security Group (`smartqna-alb-sg`):

- Inbound: TCP port 443 from `0.0.0.0/0` (all HTTPS traffic)
- Outbound: All traffic

EC2 Security Group (`smartqna-ec2-sg`):

- Inbound: TCP port 80 from `smartqna-alb-sg` only (not the internet)
- Outbound: All traffic

RDS Security Group (`smartqna-rds-sg`):

- Inbound: TCP port 5432 from `smartqna-ec2-sg` only
- Outbound: All traffic

> This layered approach ensures that EC2 is never directly exposed to the internet, and RDS is only reachable from EC2.

Step 2 — AWS Cognito (Authentication)

1. Go to **Cognito** → **User Pools** → **Create user pool**
2. **Pool name:** `smartqna-users`
3. Sign-in option: Email
4. Create an **App Client**: `smartqna-web`
    - Auth flows: `ALLOW_USER_PASSWORD_AUTH`, `ALLOW_REFRESH_TOKEN_AUTH`
    - No client secret
5. Note the **User Pool ID** and **App Client ID**
6. Create three **Groups** inside the User Pool:
    - `STUDENT` — default role for new users
    - `TA` — teaching assistant role
    - `ADMIN` — full admin access

> Cognito groups are embedded in the JWT as `cognito:groups`. Your backend's `require_role()` function reads this claim to enforce access control.

Step 3 — Amazon RDS (Database)

1. Go to **RDS** → **Create database** → **Standard create**
2. Engine: **PostgreSQL 15**
3. Template: Free tier (or db.t3.micro)
4. DB instance identifier: `smartqna-db`
5. Set master username and password (save these securely)
6. Assign the **RDS security group** created in Step 1
7. Disable public access

8. Note the **RDS endpoint** after creation (e.g. `smartqna-db.xxxx.us-east-1.rds.amazonaws.com`)

After RDS is running, connect to it and run:

```
-- Enable pgvector for AI similarity search
CREATE EXTENSION IF NOT EXISTS vector;
```

Run database migrations from an EC2 instance after setup:

```
cd /home/ec2-user/smartqna
DATABASE_URL=postgresql+asyncpg://<user>:<pass>@<rds-endpoint>:5432/smartqna \
  docker compose -f docker-compose.prod.yml exec backend alembic upgrade head
```

Step 4 — Amazon S3 (Two Buckets)

Bucket 1 — Frontend Hosting:

- Name: `smartqna-frontend`
- Region: `us-east-1`
- Block all public access: **ON** (CloudFront will be the only accessor via OAC)
- Enable static website hosting: `index.html` as both index and error document

Bucket 2 — File Attachments:

- Name: `smartqna-attachments`
- Region: `us-east-1`
- Block all public access: **ON**
- Files are accessed exclusively through pre-signed URLs generated by the backend

---

Step 5 — Amazon CloudFront (CDN)

1. Go to **CloudFront** → **Create distribution**
2. **Origin:** `smartqna-frontend` S3 bucket
3. **Origin Access:** Create new **Origin Access Control (OAC)** — keeps S3 private
4. **Default root object:** `index.html`
5. **Custom error responses:**
   - HTTP 403 → `/index.html` → Response code 200
   - HTTP 404 → `/index.html` → Response code 200

   > This is required so React Router handles client-side routing correctly (e.g., refreshing `/posts/123` doesn't return a 404)

6. Note the **CloudFront domain name** (e.g. `d1abc123.cloudfront.net`) — this is your public app URL
7. Note the **Distribution ID** — needed for GitHub Secrets

Step 6 — Amazon ECR (Docker Registry)

1. Go to **ECR** → **Create repository** → **Private**

2. Repository name: `smartqna-backend`
3. Note the **full repository URI**:
   `<account-id>.dkr.ecr.us-east-1.amazonaws.com/smartqna-backend`

GitHub Actions will push Docker images here. EC2 instances will pull from here on every deploy.

Step 7 — EC2 Instances (Backend Servers)

Launch 2 EC2 instances (one at a time):

- AMI: Amazon Linux 2023
- Instance type: t3.micro
- Key pair: Create or reuse an existing key pair (save the `.pem` file)
- Security group: `smartqna-ec2-sg`
- IAM Instance Profile: attach a role with these permissions:
  - `AmazonEC2ContainerRegistryReadOnly` — to pull Docker images from ECR
  - `AmazonSQSFullAccess` — to enqueue notifications
  - `AmazonS3FullAccess` — to generate pre-signed URLs
  - `AmazonBedrockFullAccess` — for AI embedding calls

On each EC2 instance, SSH in and run:

```
# Install Docker
sudo yum update -y
sudo yum install docker -y
sudo service docker start
sudo usermod -aG docker ec2-user

# Create app directory
mkdir -p /home/ec2-user/smartqna

# Copy docker-compose.prod.yml to this directory
# (with all placeholders filled in)
```

Place a completed `docker-compose.prod.yml` at `/home/ec2-user/smartqna/docker-compose.prod.yml` on each instance with real values for RDS endpoint, Cognito IDs, S3 bucket names, and SQS URL.

Step 8 — Application Load Balancer

1. Go to **EC2** → **Load Balancers** → **Create Application Load Balancer**
2. **Name:** `smartqna-alb`
3. **Scheme:** Internet-facing
4. **Listeners:** HTTPS (port 443)
5. **SSL certificate:** Request a free certificate from **AWS Certificate Manager (ACM)**
6. **Security group:** `smartqna-alb-sg`

Create a Target Group:

- Name: `smartqna-targets`
- Target type: Instances
- Protocol: HTTP, Port: 80

- Health check path: `/api/v1/health`
- Register both EC2 instances

> The ALB continuously pings `/api/v1/health` on each instance. If an instance becomes unhealthy, the ALB stops sending traffic to it automatically.

Step 9 — Amazon SQS (Notification Queue)

1. Go to **SQS** → **Create queue** → **Standard Queue**
2. **Name:** `smartqna-notifications`
3. Default settings are fine for a class project
4. Note the **Queue URL** (e.g. `https://sqs.us-east-1.amazonaws.com/<account-id>/smartqna-notifications`)

Step 10 — AWS Lambda (Notification Worker)

1. Go to **Lambda** → **Create function** → **Author from scratch**
2. **Name:** `smartqna-notification-worker`
3. **Runtime:** Python 3.11
4. Upload the code from `lambda/notification_worker/handler.py`
5. Add a **layer** with `psycopg2-binary` and `boto3` dependencies
6. Set environment variables:
   - `DB_HOST` — RDS endpoint
   - `DB_NAME` — `smartqna`
   - `DB_USER` — your RDS username
   - `DB_PASSWORD` — your RDS password
7. **Add trigger** → **SQS** → select `smartqna-notifications` queue
8. Batch size: 10 (Lambda processes up to 10 messages per invocation)

> Lambda wakes up automatically whenever EC2 enqueues a message to SQS. It writes the notification record to RDS. No always-on server needed.

Step 11 — Amazon Bedrock (AI Similarity Search)

1. Go to **Bedrock** → **Model access** → **Manage model access**
2. Enable: **Amazon Titan Embeddings G1 - Text**
3. No infrastructure to provision — Bedrock is fully serverless

> EC2 instances call Bedrock via the AWS SDK when a user submits or previews a question. Bedrock returns a 1536-dimension vector representing the semantic meaning of the text. This vector is then compared against existing question vectors stored in RDS using pgvector's `<=>` cosine distance operator.

Step 12 — IAM Group and Users

1. Create an IAM group: `smart-qna-developers`
2. Attach policies: Cognito, RDS, EC2, S3, CloudFront, SQS, Lambda, Bedrock, ECR
3. Create one IAM user per teammate, add each to the group
4. Each teammate creates an Access Key for CLI use (`aws configure`)

---

5. Phase 2 — CI/CD Automated Deployments

Once Phase 1 is complete, all future deployments are **fully automatic**.

Backend Deployment Flow

Triggered by: `git push origin main` with changes in `backend/`

```
Push to main (backend change)
        │
        ▼
  [Job: test]
  ├── GitHub spins up Ubuntu runner
  ├── Starts PostgreSQL 15 container (sidecar service)
  ├── Installs Python 3.11 + requirements.txt
  ├── Runs: alembic upgrade head
  ├── Runs: pytest -v
  │
  All tests pass? ──No──► Workflow FAILS. Deploy blocked.
  │ Yes
  ▼
  [Job: deploy] (only runs after test passes)
  ├── Authenticates to AWS (via GitHub Secrets)
  ├── Logs into ECR
  ├── docker build -t smartqna-backend backend/
  ├── docker tag → ECR URI:latest
  ├── docker push → ECR
  ├── SSH into EC2 #1 and EC2 #2
  └── On each EC2:
        ├── docker pull ECR:latest
        └── docker compose -f docker-compose.prod.yml up -d
              │
              ▼
        ALB health checks confirm instances are healthy
        New backend is live
```

Frontend Deployment Flow

Triggered by: `git push origin main` with changes in `frontend/`

```
Push to main (frontend change)
        │
        ▼
  [Job: deploy]
  ├── GitHub spins up Ubuntu runner
  ├── Installs Node.js 20
  ├── npm ci (exact dependency install)
  ├── npm run build → produces frontend/dist/
  │     └── VITE_API_URL injected at build time → points to ALB
  ├── Authenticates to AWS
  ├── aws s3 sync frontend/dist/ s3://smartqna-frontend --delete
  │     └── --delete removes files no longer in the build
  └── CloudFront cache invalidation (--paths "/*")
        └── All users see the new version immediately
```

Key CI/CD Properties

- **Tests are enforced** — no backend code reaches EC2 unless all pytest tests pass
- **Independent pipelines** — frontend and backend deploy separately based on which files changed
- **Atomic** — each deploy runs identically regardless of which teammate pushed
- **Auditable** — GitHub Actions tab shows full history of every deploy, who triggered it, and pass/fail status

---

6. AI Similarity Search Feature

Overview

When a user types a new question, the system checks for semantically similar existing questions before submission — similar to Stack Overflow's duplicate detection.

How It Works

```
User types question text
        │
        ▼
Browser → POST /posts/similar { "text": "How do I sort a list in Python?" }
        │
        ▼
EC2 (FastAPI)
  ├── Calls Amazon Bedrock: Titan Embeddings
  │     └── Input: question text string
  │     └── Output: 1536-dimension float vector
        │
        ▼
EC2 → RDS (pgvector)
  └── SELECT id, title, (embedding <=> $1) AS distance
      FROM posts
      ORDER BY distance ASC
      LIMIT 5
        │
        ▼
Returns top 5 semantically similar questions with similarity scores
        │
        ▼
Frontend displays: "Similar questions found — did you mean one of these?"
User can view existing answers or post anyway
```

Database Changes Required

A new Alembic migration (Migration 004) must add a vector column to the posts table:

```
ALTER TABLE posts ADD COLUMN embedding vector(1536);
CREATE INDEX ON posts USING ivfflat (embedding vector_cosine_ops);
```

Why Bedrock + pgvector Over Alternatives

Option	Pros	Cons
Bedrock + pgvector ☑	Reuses existing RDS, low cost, simple setup	Limited to ~100k posts before performance tuning needed
Bedrock + OpenSearch	Scales to millions of posts	~$700/month minimum cost
Keyword search (existing)	Already implemented	Cannot detect semantic similarity

7. Environment Variables Reference

These are set in `docker-compose.prod.yml` on each EC2 instance:

Variable	Description	Example
`DATABASE_URL`	Full asyncpg connection string to RDS	`postgresql+asyncpg://user:pass@endpoint:5432/smartqna`
`SECRET_KEY`	Application secret for signing	A long random string
`ENVIRONMENT`	Runtime environment flag	`production`
`CORS_ORIGINS`	Allowed frontend origins	`["https://d1abc.cloudfront.net"]`
`AWS_REGION`	AWS region	`us-east-1`
`COGNITO_USER_POOL_ID`	Cognito User Pool ID	`us-east-1_XXXXXXX`
`COGNITO_APP_CLIENT_ID`	Cognito App Client ID	`xxxxxxxxxxxxxxxxx`
`S3_BUCKET_ATTACHMENTS`	S3 bucket name for file uploads	`smartqna-attachments`
`SQS_NOTIFICATION_QUEUE_URL`	Full SQS queue URL	`https://sqs.us-east-1.amazonaws.com/xxxx/smartqna-notifications`

8. GitHub Secrets Reference

Set these at: **GitHub repo → Settings → Secrets and variables → Actions**

Secret Name	Description	Where to Find

Secret Name	Description	Where to Find
`AWS_ACCESS_KEY_ID`	IAM user access key ID	AWS Console → IAM → Users → Security credentials
`AWS_SECRET_ACCESS_KEY`	IAM user secret access key	Shown once at key creation
`ECR_REPO`	Full ECR repository URI	AWS Console → ECR → your repo → URI
`EC2_HOST`	ALB DNS name	AWS Console → EC2 → Load Balancers → DNS name
`EC2_SSH_KEY`	Private SSH key for EC2 access	The `.pem` file content from your EC2 key pair
`S3_FRONTEND_BUCKET`	Frontend S3 bucket name	`smartqna-frontend`
`CF_DISTRIBUTION_ID`	CloudFront distribution ID	AWS Console → CloudFront → your distribution → ID

9. Deployment Timeline

Week 1 — Infrastructure

Day	Tasks
Day 1	Create Security Groups · Set up Cognito User Pool + App Client + Groups
Day 2	Launch RDS · Enable pgvector · Run Alembic migrations
Day 3	Create S3 buckets (frontend + attachments) · Set up CloudFront distribution
Day 4	Create ECR repository · Launch EC2 #1 and EC2 #2 · Install Docker on both
Day 5	Create ALB + Target Group · Verify health checks pass

Week 2 — Services and CI/CD

Day	Tasks
Day 1	Create SQS queue · Deploy Lambda function · Enable Bedrock model access
Day 2	Add all GitHub Secrets · Push backend code → verify CI/CD pipeline end to end
Day 3	Push frontend code → verify S3 sync + CloudFront invalidation
Day 4	Full user flow test: register → login → post → similar questions → notification
Day 5	Bug fixes · Final review · Add teammates to IAM group

10. Full Service Map

```
USER BROWSER (React 18 SPA)
│
├──[Login]──────────────────────────────► AWS COGNITO
│                                          └── Issues JWT token
│◄──────────────────────────── JWT token ──────────────────────────
│
```

```
├──[API calls + JWT]──────────────────────► AMAZON CLOUDFRONT
│                                            ├── Serves React app from S3
│                                            └── Routes /api/v1 to ALB
│
│                              AMAZON S3 (Frontend) ◄── GitHub Actions syncs here
│
│                              APP LOAD BALANCER
│                               ├──► EC2 #1 (FastAPI + Docker)
│                               └──► EC2 #2 (FastAPI + Docker)
│                                      │
│                                      ├──[verify JWT]──► COGNITO JWKS (once, cached)
│                                      │
│                                      ├──[SQL]──────────► AMAZON RDS (PostgreSQL +
pgvector)
│                                      │                    └── Stores posts, answers,
users,
│                                      │                        notifications, embedding
vectors
│                                      │
│                                      ├──[pre-sign]──────► AMAZON S3 (Attachments)
│                                      │                    └── User uploads files
directly
│                                      │
│                                      ├──[embedding]─────► AMAZON BEDROCK
│                                      │                    └── Returns 1536-dim vector
│                                      │                    └── EC2 queries RDS for
similar posts
│                                      │
│                                      └──[enqueue]───────► AMAZON SQS
│                                                           └──[trigger]──► AWS LAMBDA
│                                                                            └── INSERT
notifications → RDS
│
CI/CD PIPELINE (GitHub Actions)
├── Backend: test → build → push ECR → SSH deploy to EC2 #1 & #2
└── Frontend: build → sync S3 → invalidate CloudFront
```

Checklist — Ready to Deploy?

Infrastructure

- ☐ Security groups created (ALB, EC2, RDS)
- ☐ Cognito User Pool + App Client + Groups (STUDENT, TA, ADMIN) created
- ☐ RDS instance running, `pgvector` extension enabled, migrations run
- ☐ S3 frontend bucket created with static hosting
- ☐ S3 attachments bucket created
- ☐ CloudFront distribution created with OAC and error page rules
- ☐ ECR repository created
- ☐ EC2 #1 launched, Docker installed, `docker-compose.prod.yml` placed
- ☐ EC2 #2 launched, Docker installed, `docker-compose.prod.yml` placed
- ☐ ALB created, Target Group registered both EC2 instances, health checks passing
- ☐ SQS queue created
- ☐ Lambda function deployed with SQS trigger

- ☐ Bedrock Titan Embeddings model access enabled
- ☐ IAM group created, all teammates added

CI/CD

- ☐ All 7 GitHub Secrets added to repository
- ☐ SSH public key added to both EC2 instances
- ☐ Backend pipeline tested: push → tests pass → ECR push → EC2 deploy
- ☐ Frontend pipeline tested: push → build → S3 sync → CloudFront invalidation

Functional Testing

- ☐ User can register and login via Cognito
- ☐ User can post a question
- ☐ Similar question suggestions appear before posting (Bedrock + pgvector)
- ☐ User can submit an answer and vote
- ☐ In-app notification appears after an answer is posted (SQS → Lambda → RDS)
- ☐ File attachment upload works (S3 pre-signed URL)
- ☐ TA/Admin role restrictions enforced (pin, close, delete)